

Solving Sudoku: From Generic Backtracking to CSP Optimizations

Sitan Chen

1. Introduction

The inspiration for this project stemmed from a summer school course on solving logic puzzles, such as Einstein's Riddle, using Prolog. The solution to Einstein's Riddle relies on search and backtracking algorithms. This experience sparked the realization that similar algorithmic approaches could be effectively applied to other logic puzzles, notably Sudoku. Moreover, it highlighted opportunities for algorithmic enhancement, particularly in reducing time complexity. This project investigates that potential by implementing and comparing two distinct strategies for solving Sudoku: a generic backtracking algorithm and an optimized approach that incorporates Constraint Satisfaction Problems (CSP) techniques.

2. Background

2.1 Sudoku

As a globally popular logic puzzle, Sudoku is governed by seemingly simple rules: the objective is to fill a 9×9 grid so that each digit from 1 to 9 appears exactly once in every row, column, and 3×3 subgrid. Despite their simplicity, these combinatorial uniqueness constraints define a challenging problem space, classifying Sudoku as a Constraint Satisfaction Problem (CSP), in which valid solutions must simultaneously satisfy all given constraints.

2.2 CSP Optimization

CSP optimization refers to a set of techniques designed to enhance the efficiency of solving Constraint Satisfaction Problems. A Constraint Satisfaction Problem (CSP) is defined by three key components: variables (e.g., empty cells in a Sudoku grid), domains (e.g., the set of possible digits {1–9} for each cell), and constraints (e.g., the requirement that each digit appears exactly once in every row, column, and 3×3 subgrid).

A common CSP optimization is achieved through two core strategies. First, the Minimum Remaining Values (MRV) heuristic guides the search by selecting the variable with the fewest valid values remaining, thereby increasing the likelihood of early detection of conflicts and pruning futile branches sooner. Second, constraint propagation dynamically removes inconsistent values from the domains of related variables after each assignment, effectively narrowing the search space and preventing invalid configurations.

3. Methodology

3.1 Backtracking

Backtracking is a fundamental depth-first search (DFS) algorithm widely used in decision-based problem solving. It systematically explores potential solutions by incrementally building candidates and backtracking (reversing previous choices whenever a partial assignment violates the problem's constraints). The core procedure follows these steps:

- **Stepwise Assignment:** Construct a solution by assigning values to variables one at a time, in a predetermined order.
- **Consistency Validation:** After each assignment, check whether the current state satisfies all constraints.
- **Backtracking Trigger:** If no valid value can be assigned to the current variable, undo the most recent assignment and explore alternative values for that variable.
- **Termination:** Repeat this process until either a complete and valid solution is found or all possible assignments have been exhausted.

Due to its systematic exploration and pruning of invalid paths, backtracking is particularly well-suited for problems involving combinations, permutations, and constraint satisfaction, including Sudoku.

In the context of Sudoku, the algorithm begins at the top-left cell of the 9×9 grid and proceeds row by row, from left to right and top to bottom. For each empty cell, it attempts digits from 1 to 9 in sequence, checking whether each candidate value adheres to the Sudoku constraints (i.e., no duplicates in the corresponding row, column, and 3×3 subgrid). If no valid digit can be placed in a cell, the algorithm concludes that an earlier assignment must be incorrect and backtracks to revise previous choices. This iterative process continues until a complete and consistent solution is achieved.

3.2 Disadvantages of Backtracking

Although backtracking solves Sudoku puzzles in a straightforward and intuitive manner, it suffers from a critical drawback: an extremely high time complexity of $O(9^n)$, where n is the number of empty cells. This inefficiency arises because the algorithm does not fully leverage the inherent structure of Sudoku's constraints. Specifically, backtracking only applies the rules—such as the requirement for unique digits in each row, column, and 3×3 subgrid—to validate a guess after it has been made, rather than using them proactively to guide and prune the search space.

For example, if the digit 4 already appears in row 1, it is impossible for 4 to appear again in any other empty cell of that row. However, standard backtracking does not

eliminate 4 from the candidate list for those cells in advance, potentially leading to redundant and invalid guesses. By failing to incorporate constraint information earlier in the process, the algorithm wastes time exploring paths that could have been ruled out immediately. This highlights a key opportunity for optimization through more intelligent use of the puzzle's constraints during the search.

3.3 Introducing MRV and Constraint Propagation

To enhance the efficiency of backtracking, we incorporate Constraint Satisfaction Problem (CSP) optimization techniques. First, we apply the Minimum Remaining Values (MRV) heuristic, which selects the next empty cell to assign based on the fewest number of valid candidate values. This strategy helps to quickly expose dead ends and prune unproductive branches of the search tree early. Second, we implement constraint propagation, which actively enforces Sudoku's core constraints, namely, the requirement that each digit from 1 to 9 must appear exactly once in every row, column, and 3×3 subgrid.

4. Implementation

4.1 Implementation of Backtracking

The backtracking implementation consists of a main loop that systematically assigns valid numbers to empty cells while maintaining Sudoku's constraints. When the algorithm encounters a cell for which no valid digit can be placed, it backtracks to the most recently filled cell and attempts the next available number in its candidate list. This process continues recursively until a complete and valid solution is found. Algorithm 1 illustrates the implementation of the backtracking algorithm.

Algorithm 1 Backtracking Algorithm for Sudoku

Input: A 9x9 Sudoku grid, where empty cells are represented by 0.

Output: Returns true if the Sudoku can be solved, false otherwise; the solved grid is modified in place

```

1. procedure Solve(grid)
2.   if (!find_empty(grid, row, col)) //If there is no empty cell in the grid, the puzzle is solved
3.     return true
4.   for num ← 1 to 9 do // Try to fill each empty cell with numbers from 1 to 9
5.     if is_valid(row, col, num) == true //Checks whether placing num satisfies the constraints
6.       grid[row][col] ← num //Commits the valid digit into the grid
7.       if Solve(grid) //Recursively invokes Solve() to solve the rest of the Sudoku
8.         return true
9.       grid[row][col] ← 0 // Undoes the last placement by resetting the cell to empty
10.    end for
11.    return false // Trigger backtracking
12. end procedure

```

4.2 Implementation of MRV and Constraint Propagation

Building upon the backtracking logic, the enhanced version integrates Constraint Propagation and the Minimum Remaining Values (MRV) heuristic through the addition of two key functions to optimize the solving process.

First, the algorithm calls the *propagate_constraints()* function, which proactively eliminates values from the domains of unassigned cells by enforcing Sudoku's constraints, thereby removing assignments that would lead to contradictions.

Next, the solver invokes the *find_mrv_cell()* function to select the empty cell with the fewest remaining valid candidates. This strategic choice increases the likelihood of either advancing the solution efficiently or uncovering conflicts early, minimizing unnecessary exploration.

When assigning a value to a selected cell, the algorithm iterates only over the numbers in the candidate list (those that are still valid under current constraints) rather than blindly testing digits from 1 to 9. If no valid value can be assigned to a cell, the algorithm backtracks to the most recently assigned cell and tries the next available option. Algorithm 2 illustrates the implementation of this optimized version, combining constraint propagation and MRV.

Algorithm 2 CSP Optimization Algorithm for Sudoku

Input: global *grid*[*N*][*N*]

Output: returns true if the grid is solved, false otherwise. grid is modified in place

```

1. procedure Solve_optimization()
2.     if propagate_constraints() = false    //Run constraint propagation to remove impossible values
3.         return 0                          //If a contradiction arises, abort
4.     if grid has no EMPTY cell            //The puzzle is solved
5.         return 1
6.     if (!find_mrv_cell(row, col))        //MRV selection
7.         return 0                          //No legal value left
8.     saved ← copy of every cell's candidates    //Save state for backtracking
9.     for each candidate value v of grid[row][col]    //Try each legal value in the cell
10.         grid[row][col].value ← v
11.         if Solve_optimization() = 1
12.             return 1
13.     end for
14.     grid[row][col].value ← EMPTY    //On returning from recursion, undo the assignment
15.     restore all candidates from saved
16.     return 0    //If no value works, trigger backtracking
17. end procedure

```

5. Experimental Evaluation

Below, we evaluate the performance of the implemented backtracking algorithm and the optimized version incorporating MRV and constraint propagation. We selected Sudoku puzzles of seven different difficulty levels from the website

<https://www.menneske.no/sudoku/eng/random.html>. The experiments were conducted on a MacBook M4 Pro, with the code compiled using clang and the O2 optimization flag. For each algorithm, we performed 7 independent runs and recorded the average execution time (in microseconds).

As shown in Table 1, the optimized version with MRV and constraint propagation improves the performance up to 15x (from 1.5x to 15x). As shown in Figure 1, regarding backtracking, except for the "Hard" difficulty level, its processing time for Sudoku puzzles shows an upward trend with the increase in difficulty. As for the optimized version, except for the "Impossible" difficulty level, its processing time tends to be stable. For the "Hard" and "Super hard" difficulty level, the optimized version shows superior performance as compared to basic backtracking. For the "Impossible" difficulty level, a 1.49x improvement can be observed.

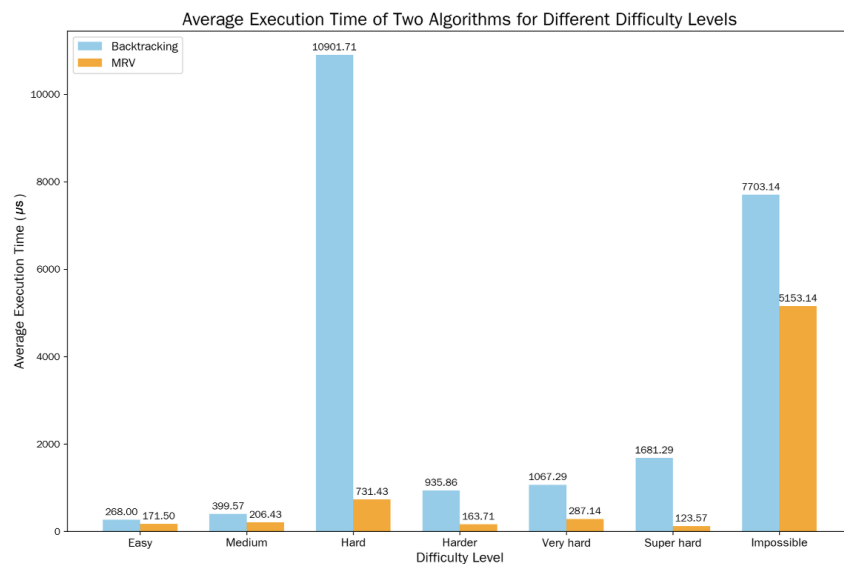


Figure 1 Comparison of two algorithms across 7 difficulty levels.

Initially, we attempted to control the difficulty of Sudoku puzzles by limiting the number of given clues (i.e., pre-filled cells). However, in subsequent experiments, the resulting data failed to align with basic expectations. We then realized that the number of given clues alone is not a reliable indicator of puzzle difficulty. We therefore hypothesize that difficulty is also influenced by other factors, such as the spatial arrangement of the given numbers. Moreover, different solving algorithms may exhibit varying performance depending on the structure and pattern of the puzzle. Some arrangements may be inherently more challenging or favorable for a specific algorithm. This suggests that the anomalous results observed at the "Hard" difficulty level could stem from such structural variations, rather than the sheer number of clues.

Table 1. Speedups of MRV and constraint propagation over backtracking.

	Easy	Medium	Hard	Harder	Very hard	Super hard	Impossible
Backtracking	268.00	399.57	10901.7	935.86	1067.29	1681.29	7703.14
MRV & Constraint Propagation	171.50	206.43	731.43	163.71	287.14	123.57	5153.14
Speedup	1.56x	1.94x	14.9x	5.72x	3.72x	13.61x	1.49x

6. Conclusion

This project demonstrates that integrating CSP techniques, such as MRV heuristics and constraint propagation, into backtracking significantly enhances Sudoku-solving efficiency, achieving up to 15× speedups. Optimized algorithms excel in complex puzzles by proactively pruning search spaces. Results also reveal that puzzle difficulty correlates not solely with clue count but also structural factors, necessitating future research on accurate benchmarking.